DOMAIN 3 // VULNERABILITY DISCOVERY & ANALYSIS

COMPTIA PENTEST+ PT0-003 · OBJECTIVES V3.0 · FULL REFERENCE

17% OF EXAM · ~15 QUESTIONS

TWO ORGANIZING IDEAS — READ THIS FIRST

- **Match the scan to the artifact.** Source code, a running application, a container image, a host, a network, a wireless environment, a repository — each has its own scan type. 3.1 is almost entirely this mapping, plus one cross-cutting question: *how much access do you have?*
  EXPECT Read the scenario for *what is being pointed at* and *whether source or credentials are available*. Those two facts decide nearly every 3.1 answer.
- **Scanner output is a hypothesis, not a finding.** 3.2 is the bridge between "the tool said something" and "the report says something". Unvalidated scanner output in a deliverable is exactly what quality control in 1.4 exists to catch.
  EXPECT When a question presents scan results, the answer is usually to *verify* before acting — not to escalate severity, not to report as-is.
- **Physical access ends the technical argument.** 3.3 is the layer where encryption and endpoint controls buy time rather than immunity.
  EXPECT 3.3's verb is *explain*, not "given a scenario" — definitional questions, not multi-step reasoning. Know what each concept is, what it defeats, and which 1.5 control stops it.

3.1 VULNERABILITY DISCOVERY — GIVEN A SCENARIO, CONDUCT

APPLICATION SCANS — THE HIGHEST-YIELD CLUSTER IN DOMAIN 3

TYPE	ANALYZES	NEEDS IT RUNNING	WHAT TO EXPECT ON THE EXAM
SAST	Source, bytecode or binaries. Sub-bullets: **IaC** (Terraform, CloudFormation, K8s manifests) and **source code analysis**	No	"Earliest in the SDLC", "before deployment", "identify the exact line". Any Terraform / CloudFormation / manifest mention. High false-positive rate — no runtime context to prove the path is reachable
DAST	The deployed application from the outside; no source access	Yes	"No source code available", "third-party application we did not write", "test the staging deployment". Findings are usually real but carry no line-of-code pointer
IAST	The running application from inside, via an agent, while it is exercised	Yes	"Agent", "instrumented", "while functional tests run". Highest accuracy of the four — code location *and* runtime confirmation. Requires an agent in the runtime; language-dependent
SCA	Third-party and open-source dependencies matched to known CVEs and licence issues	No	"Vulnerable library", "open-source component", "transitive dependency". The answer whenever the vulnerable code was *not written by the client*. Source of the SBOM concept

- **Memory hook** — SAST reads the code, DAST pokes the app, IAST rides along inside, SCA checks whose code you shipped.
- **Mobile scan** — analysing APK/IPA packages for hardcoded secrets, insecure local storage, weak crypto, exported components, missing certificate pinning. Static and dynamic both apply.
  EXPECT Tools live in 4.9 (MobSF, Frida, Drozer, ADB), not here. This bullet is about the scan category.

CONTAINER SCANS

- **Image scanning** — layer-by-layer inspection for vulnerable OS packages and app dependencies, misconfiguration, embedded secrets, running as root, unnecessary exposed ports.
  EXPECT A vulnerability in the *base image* propagates to every image built on it — that makes it a *process-level remediation* problem from 1.5, not a per-image fix.
- **Sidecar scans** — a helper container in the same pod, sharing the network namespace and volumes, inspecting the workload at runtime.
  EXPECT "Scan the running workload without modifying the application image" → sidecar. Second angle: sidecars themselves (service-mesh proxies, log shippers) are overlooked attack surface inheriting the main container's access.

NETWORK, HOST AND ACCESS LEVEL

- **Network scans** — TCP/UDP as covered in 2.1.
- **Stealth scans** — FIN, NULL, Xmas. No SYN flag; RFC 793 says a closed port must reply RST and an open port must stay silent. **Silence is the interesting result** — the inverse of a normal scan.
  EXPECT Windows does not follow the RFC and returns RST for every port. A FIN/NULL/Xmas scan reporting *everything closed* against evidence to the contrary means the target is Windows.
- **Host-based scans** — agent or credentialed check reading patch level, registry, installed software, local accounts, file permissions, endpoint control status.
  EXPECT Network scan sees what is exposed on the wire; host-based sees internal state. A host can look clean from the network and be badly out of date inside.
- **Authenticated scans** — the scanner logs in and reads installed package and patch state directly. Far fewer false positives *and* false negatives; finds local-only issues. Cost: credential provisioning and the risk those credentials carry.
  EXPECT Back-ported patches leave the version banner unchanged, so an unauthenticated scan reports a vulnerability an authenticated scan correctly clears. This is the standard false-positive setup.
- **Unauthenticated scans** — outside view only, inferring from banners and behaviour.
  EXPECT "Represent the external attacker's initial view" → unauthenticated. "Reduce false positives" or "accurate patch data" → authenticated.
- **Secrets scanning** — repos, filesystems, images and CI config for API keys, cloud access keys, private keys, tokens. Regex for known key formats plus entropy analysis.
  EXPECT *Commit history must be included* — a secret deleted in HEAD stays retrievable. Remediation: rotate first, then secrets management (1.5).

WIRELESS SCANS AND ICS ASSESSMENT

- **SSID scanning** — discovering networks including hidden ones.
- **Channel scanning** — channels in use, overlap, rogue APs on unexpected channels.
- **Signal strength scanning** — coverage mapping.
  EXPECT The non-obvious one. Its security relevance is the *physical boundary*: signal bleeding into a parking lot or adjacent tenant lets an attacker work from outside the controlled area. Also how you triangulate a rogue AP.
- **ICS — manual assessment** — reviewing configurations, firmware versions, architecture and vendor advisories by hand instead of running a scanner, because active scanning can crash a PLC and a crashed PLC is a *safety* event.
  EXPECT Any ICS / SCADA / OT / plant scenario → manual assessment plus passive capture. "Run a credentialed Nessus scan against the PLC segment" is always wrong. Ties to Purdue L0/L1 in 1.3 and exclusions in 1.1.

- **ICS — port mirroring** — SPAN port copying traffic to an analysis interface so OT protocols can be inspected without injecting anything.

EXPECT The OT-safe substitute for active enumeration. Industrial protocols are cleartext by design, so passive capture yields register values directly.

TOOLS

TOOL	SCANS WHAT	WHAT TO EXPECT ON THE EXAM
Nikto	Web servers — dangerous files, outdated software, default content, misconfiguration	Web-server-specific and deliberately noisy; never the stealthy option
Greenbone / OpenVAS	General network and host vulnerabilities	The open-source general-purpose scanner
Tenable Nessus	General network and host vulnerabilities; plugin-based, credentialed and compliance scans	The commercial general-purpose scanner
TruffleHog	Secrets in repositories and commit history	The secrets-scanning answer
BloodHound	Active Directory relationships and attack paths	Maps paths, does not scan for CVEs. Also in 2.2 and 4.3
PowerSploit	Windows hosts — local privilege-escalation opportunities, post-exploitation	PowerShell offensive framework; also 4.10
Grype	Container images and filesystems — package vulnerabilities	Focused image / filesystem vulnerability matching
Trivy	Images, filesystems, repositories, IaC, secrets, Kubernetes	The broad one — pick it when a scenario spans several artifact types
Kube-hunter	Kubernetes clusters — misconfiguration and attack surface	Cluster level, not image level. Also 4.6

3.1 DIFFERENTIATORS

SAST vs DAST — reads code without running it vs tests the running app with no source.
IAST vs both — agent inside the runtime gives code location *and* proof the path is reachable.
SCA vs SAST — code you imported vs code you wrote.
Container image scan vs sidecar scan — image contents before/outside runtime vs the live workload without touching the image.
Network scan vs host-based scan — exposed on the wire vs the system's internal state.
Authenticated vs unauthenticated — reads installed state vs infers from banners; the former cuts both error types.
Grype vs Trivy — image and filesystem package matching vs images + IaC + secrets + repos + Kubernetes.
Trivy/Grype vs Kube-hunter — what is inside the images vs how the cluster is configured.
Nikto vs Nessus — web-server-specific vs general purpose.

3.2 ANALYSING OUTPUT — GIVEN A SCENARIO, ANALYZE

THE FOUR OUTCOMES

OUTCOME	MEANING	WHAT TO EXPECT ON THE EXAM
True positive	Reported and real	The goal state. Confirm it before it reaches the report — confirmation is not the same as exploitation
False positive	Reported, not real	Costs credibility and wastes the client's remediation budget. Visible and embarrassing, but it fails *safe* — someone investigates and closes it
False negative	Real but missed	**The dangerous one.** Creates false assurance: the client allocates no effort and the exposure stays live. Fails silently
True negative	Correctly absent	Not in the official bullets, but completes the matrix and helps you reason about the other three

- **Why false positives happen**
  CAUSES Back-ported patch leaves the banner unchanged · version matches a CVE but the vulnerable module is not enabled · a compensating control already blocks exploitation · generic or buggy plugin · flagged default account is actually disabled · plugin fires against the wrong platform
- **Why false negatives happen**
  CAUSES Unauthenticated scan cannot see local state · IPS blocked or throttled the scanner mid-run · port or protocol never scanned (UDP skipped, non-standard port) · host offline during the window · CREDENTIALS FAILED SILENTLY · stale plugin feed · load balancer meant one backend node was reached · web app required login and the scanner never authenticated
  EXPECT The silent credential failure is the highest-value item on that list. The standard setup is a scan that "completed successfully" with a suspiciously clean result.
- **Scan completeness** — hosts scanned vs hosts in scope · port coverage (nmap defaults to top 1000 TCP; `-p-` for all 65535; UDP needs `-sU`) · authentication success rate · whether the scan was terminated, timed out or blocked · whether the scanner's vantage point could reach every segment.
  EXPECT "The scan completed successfully" is not "the scan was complete". Watch for a host count below the target list, zero authenticated hosts, or a finding count that dropped sharply since the last run.
- **Troubleshooting scan configurations** — in checking order: credentials (wrong account, insufficient privilege, lockout triggered, Windows remote UAC) → network path (ACL, wrong VLAN) → IPS/IDS blocking → timing too aggressive (`-T5`) → host discovery skipping non-ICMP hosts (`-Pn`) → port range too narrow → stale feed → DNS resolution failures.
  EXPECT Zero findings on a known-vulnerable host is a configuration problem, not a security success. Allow-listing the scanner is correct for a vulnerability assessment and *wrong* for a red team — a 1.1 scoping decision.

PUBLIC EXPLOIT SELECTION AND SCRIPTED VALIDATION

- **Selection criteria** — exact match on product, version, architecture, build and service pack · reliability (memory-corruption exploits are often one-shot and crash the service on failure) · source trustworthiness · prerequisites (credentials, local foothold, configuration) · authorisation for anything with DoS potential · whether a lower-risk path reaches the same objective.
  EXPECT "Before running a public exploit" always means REVIEW THE CODE FIRST — public PoC code routinely contains backdoors or destructive payloads. Second most common tell: an unstable exploit against production → schedule it, authorise it, or choose a safer path.
- **Prioritisation metrics** (from 4.1) — **CVE** identifies, **CWE** classifies the weakness type, **CVSS** scores severity, **EPSS** estimates real-world exploitation probability.
  EXPECT EPSS plus public exploit availability is what tells you which confirmed findings to pursue first — severity alone does not.
- **Use scripting to validate results** — re-test a flagged finding directly (socket connect, `curl`, `openssl s_client`) · query a version endpoint across hundreds of hosts · parse scanner CSV/XML/JSON and cross-reference a second scanner · deduplicate and normalise findings before they reach the report.
  EXPECT Scripting appears here as *validation*, not exploitation. The stem usually contains "confirm", "verify", or a large host count. Draws directly on the 2.3 constructs.

3.2 DIFFERENTIATORS

False positive vs false negative — credibility and wasted effort vs false assurance. The second is the dangerous one.
Scan completeness vs troubleshooting configurations — the coverage outcome you observe vs diagnosing the settings that caused it.
Validating vs exploiting — confirming the vulnerability exists vs triggering it to demonstrate impact. On production, choose confirmation where it is sufficient.
Public exploit selection (3.2) vs capability selection (4.1) — choosing a specific exploit for a confirmed finding vs planning tooling and attack path overall.

3.3 PHYSICAL SECURITY — EXPLAIN

Chain: **site survey** (map before you act) → **gain entry** (tailgate, clone, pick) → **deliver payload** (USB drops). USB drops are the exception — they deliver a payload *without* requiring entry, because an employee carries it inside for you.

- **Site surveys** — entry and exit points including loading docks and smoking areas, badge reader types and placement, door hardware, guard posts and patrol patterns, shift change times, camera coverage gaps, fence lines and vehicle gates, visitor procedures, dumpster locations, wireless bleed beyond the property line. Plus the OSINT half: satellite and street imagery, floor plans in permit filings, employee photos showing badge design and interiors.
  EXPECT "Before attempting entry", "identify the best entry point", "determine guard rotation". Legal angle: a survey still needs RoE coverage — loitering or crossing a property line gets you detained. Carry the authorisation letter *during the survey*, not just the entry.
- **Tailgating** — following an authorised person through a controlled door without presenting credentials. Hands full with boxes or coffee, dressing the part, joining a group at shift change, re-entering from the smoking area.
  EXPECT Remediation pairing is the tested part: ACCESS CONTROL VESTIBULE prevents (physical, 1.5), USER TRAINING supports (operational, 1.5). Also appears in 4.8 as a social-engineering type.
- **Badge cloning** — copying an RFID/NFC credential. Legacy 125 kHz proximity cards transmit a static identifier with no authentication or encryption, so a single read reproduces them. Concealed long-range readers work from a metre or more; write to a blank and walk in.
  EXPECT "A cloned badge granted entry" → BIOMETRIC CONTROLS (1.5), because a biological factor cannot be captured and replayed. Encrypted credentials and shielding sleeves are the supporting fixes. RFID/NFC also appear in 4.9.
- **Lock picking** — manipulating a pin-tumbler mechanism with picks and a tension wrench, by single-pin picking or raking. **Bypass** is often more practical: shimming a padlock, slipping a latch with a shim card, an under-door tool reaching an interior lever handle, crash-bar manipulation, duplicating a key from a photograph.
  EXPECT A legal question. Possession of picks is restricted or illegal in some jurisdictions — which is why CompTIA's own hardware list says "lock pick kit (where applicable)". That is 1.1: legal and ethical considerations, and risk to the penetration tester.
- **USB drops** — devices left in parking lots, lobbies and break rooms, labelled to invite curiosity. Three classes:
  CLASSES 1. *Storage with malicious files* — macros or shortcut files; requires the user to open something. 2. *HID emulation* — declares itself a keyboard and types commands at machine speed; no user action needed. 3. *Destructive* — high-voltage discharge; essentially always excluded by RoE.
  EXPECT The HID point is the high-value one: because the device presents as a *keyboard*, autorun restrictions and removable-media policy never apply — the OS is receiving keystrokes, which is what keyboards send.

EXPECT Report the pick-up and plug-in rate as an aggregate awareness metric, never as a list of named employees — the ethical reporting consideration from 1.4. Remediation: user training (operational) plus device control, disabled autorun, and restricting unrecognised HID devices.

CROSS-REFERENCES

3.3 CONCEPT	WHERE IT REAPPEARS
Tailgating	4.8 social engineering · remediated by access control vestibule (1.5) and user training (1.5, operational)
Badge cloning	4.9 NFC and RFID attacks · remediated by biometric controls (1.5)
Site surveys	Authorisation and tester risk (1.1) · distinct from the *wireless* site survey in 2.4 / 3.1
Lock picking	Legal exposure and risk to the tester (1.1)
USB drops	Remediated by user training (1.5) · payload calls back to C2 (Domain 5)

3.3 DIFFERENTIATORS

Tailgating vs piggybacking — without the authorised person's consent vs with it (they hold the door). CompTIA often uses the terms loosely; know the distinction exists.
Physical site survey vs wireless site survey — mapping the facility vs mapping RF coverage and channels. Same phrase, different objective — read the context.
Lock picking vs bypass — manipulating the mechanism vs avoiding it entirely.
USB storage payload vs HID emulation — the user must open a file vs commands execute as keystrokes on insertion.
Preventive vs detective — vestibule and biometrics prevent; video surveillance only detects and evidences. If a question asks what would have *prevented* entry, surveillance is wrong.